

# Senior AI Engineer – Technical Assignment

Azure AI + RAG Architecture, Implementation & Problem Solving

---

## Task Overview

Design and build a **RAG-based Enterprise Knowledge Assistant** using the **Microsoft Azure AI stack**.

This assignment is intended to evaluate your ability to:

- Design production-grade AI architecture
- Build a working RAG application
- Use Azure AI services effectively
- Diagnose common RAG failures
- Improve retrieval and answer quality
- Evaluate the solution objectively
- Explain technical trade-offs and architectural decisions

We are more interested in **how you think, debug, evaluate, and improve the system** than in building a polished frontend.

You are encouraged to use AI coding tools such as **Microsoft Copilot, Cursor, Claude Code, GitHub Copilot, etc.**, but you should be able to explain and defend your implementation and architecture.

---

## Step 1 — Build a RAG Knowledge Assistant

[Here](#) is a small set of enterprise-style documents as the knowledge base.

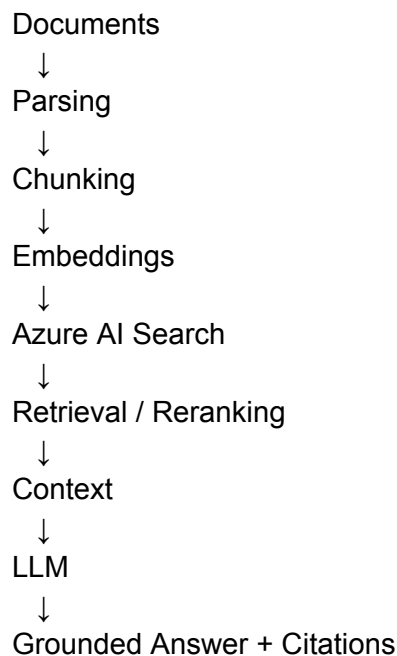
Build a chatbot that allows users to ask questions about these documents.

## Recommended Azure Stack

- Azure OpenAI
- Azure AI Search
- Azure AI Foundry
- Azure Storage
- Azure Functions / App Service
- Application Insights

Python should be used for the core application.

Your RAG pipeline should cover:



The application can have a simple UI or API. UI design is not the focus.

---

## Step 2 — Architecture Design

Prepare an architecture diagram for how you would deploy this solution for an **enterprise production environment**.

Consider:

- document ingestion

- Azure AI Search
- Azure OpenAI
- API/application layer
- authentication
- secrets management
- monitoring
- scaling
- security
- data isolation
- cost

Be prepared to explain:

Why did you choose this architecture?

Why Azure AI Search?

Semantic vs vector vs hybrid search — which would you use and why?

How would the architecture change for 10,000 documents vs 10 million documents?

## Step 3 — Solve Common RAG Failure Scenarios

We will provide **training questions and test questions**.

Some test cases are intentionally designed to make a basic RAG implementation fail.

Your job is to identify **why the system fails and improve it**.

### Scenario 1 — Correct Document, Wrong Chunk

The answer exists in the document, but retrieval returns an unrelated chunk.

Investigate:

- chunk size
- chunk overlap
- embeddings
- Top-K
- metadata filtering

- hybrid search
- reranking

Explain the root cause and implement your preferred solution.

---

## **Scenario 2 — Information Across Multiple Sections**

The answer requires information from multiple chunks or documents.

Example:

“Compare the refund policy for Enterprise and Standard customers.”

The information may exist in two different sections.

Design an approach to retrieve and combine the necessary context.

---

## **Scenario 3 — Similar Documents / Conflicting Information**

Two documents contain similar information, but one is newer.

Example:

Leave\_Policy\_2024.pdf

Leave\_Policy\_2026.pdf

The chatbot retrieves the older policy.

Design a solution using concepts such as:

- metadata
  - effective dates
  - filtering
  - ranking
  - document versioning
- 

## **Scenario 4 — Hallucination / Missing Information**

Ask a question whose answer **does not exist in the knowledge base**.

A weak chatbot may invent an answer.

Your system should detect insufficient evidence and respond appropriately rather than hallucinating.

Explain how you determine whether retrieved context is sufficient.

---

## **Scenario 5 — Ambiguous Query**

Example:

“What is the limit?”

There may be several different limits in the documents.

Determine how the chatbot should handle ambiguous questions.

Should it:

- retrieve?
- ask a clarification question?
- infer from conversation context?

Explain your decision.

---

## **Scenario 6 — Conversational Context**

Example:

User: What is the Enterprise plan cancellation policy?

User: What about Standard?

User: Is there any exception?

The system should correctly understand follow-up questions without polluting retrieval with irrelevant conversation history.

Implement an appropriate conversation-context strategy.

---



## Step 4 — RAG Evaluation

Create a small evaluation dataset.

Example:

Question

Expected Answer

Expected Document

Expected Section

Difficulty

Include approximately:

- straightforward questions
- multi-document questions
- ambiguous questions
- questions with no answer
- follow-up questions

Evaluate your system before and after your improvements.

Measure at least:

### Retrieval

- Recall / Hit Rate
- relevance of retrieved chunks

### Generation

- answer correctness
- groundedness
- citation correctness
- hallucination rate

### System

- latency
- token usage / approximate cost

Azure AI Foundry evaluation can be used, or you may implement your own evaluation approach.

Show clearly:

Baseline RAG



Identify Failures



Improve Architecture



Re-run Evaluation



Compare Results

---

## Step 5 — Architecture & Problem-Solving Questions

Include your answers to the following in your README or presentation.

### 1. Retrieval Quality

Your chatbot retrieves 5 chunks, but only one is relevant.

**How would you debug and improve it?**

### 2. Latency

Production chatbot response time increases from **3 seconds to 12 seconds**.

**How would you identify the bottleneck?**

### 3. Scale

The system grows from:

10,000 documents



5 million documents

What architectural changes would you consider?

## 4. Security

Different departments use the same chatbot:

HR  
Finance  
Legal  
Engineering

HR documents must never be retrieved for Engineering users.

**How would you architect access-controlled RAG?**

## 5. Cost

Azure OpenAI costs suddenly increase significantly.

How would you identify the cause and optimize:

- tokens
- retrieval context
- model selection
- caching
- embeddings
- repeated queries?

## 6. Production Failure

Users report:

“The chatbot gives correct answers most of the time, but occasionally gives a completely wrong answer with a valid-looking citation.”

Explain your debugging approach from:

User Query  
→ Retrieval  
→ Ranking  
→ Context  
→ Prompt  
→ LLM  
→ Citation

We care more about your **debugging methodology** than a single correct answer.



---

## Bonus

Extra marks for implementing or demonstrating:

- query rewriting
  - hybrid search
  - semantic ranking / reranking
  - metadata filtering
  - confidence scoring
  - guardrails
  - document-level access control
  - caching
  - automated RAG evaluation pipeline
  - Application Insights / production observability
- 

## Deliverables

### 1. GitHub Repository

Include:

- Python RAG application
- ingestion pipeline
- retrieval pipeline
- Azure AI Search integration
- Azure OpenAI integration
- evaluation scripts
- README

### 2. Architecture Diagram

Show the proposed **production Azure architecture**.

### 3. Evaluation Results

Show:

Baseline vs Improved RAG

and explain which changes improved which metrics.

#### 4. Demo + Architecture Presentation Video

Record a **5 minute video** covering:

1. Architecture
2. Azure services selected and why
3. Working chatbot
4. One or two RAG failure examples
5. How you diagnosed them
6. Improvements implemented
7. Evaluation before vs after
8. What you would change before production deployment

The candidate should personally explain the architecture and technical decisions.

---



## Submission Guidelines

**Email:** santosh.thota@analytos.ai

**CC:** ashok.suthar@analytos.ai

**Subject:**

Senior AI Engineer – Azure RAG Task – <Your Name>

Include:

1. GitHub repository link
  2. Architecture diagram
  3. **Demo/presentation video link (must)**
  4. Evaluation results
  5. Latest resume
- 

All the best !!